

RPAL Interpreter

Introduction

RPAL is functional programming language which doesn't contains heap allocations. This project is for implement full RPAL interpreter from scratch.

It contains the following modules,

- Scanner (Tokenize the source code)
- Parser (Parse AST)
- Analyzer (Convert AST to ST)
- CSE Machine (interpret the program)

Architecture

Entry Point

Source file: **src/main.cpp**

- Contain entry point for the program.
- Handles command line arguments.

Scanner

Source files: **src/token.hpp**, **src/token.cpp**

- Scan the source code and create linear token list

```
struct Token {
    Type type;
    const void* value;
    explicit Token(Type type);
    Token(Type type, const void* value);

    template <class T>
    T get_value() const;

    friend std::ostream &operator<<(std::ostream &os, const Token &t);
    friend bool operator==(Token const &left, Token const &right);
    friend bool operator!=(Token const &left, Token const &right);
    friend bool operator==(Token const &left, Type const &right);
    friend bool operator!=(Token const &left, Type const &right);
};
```

Parser

Source files: **src/ast.hpp** and **src/ast.cpp**

- Convert linear token list to AST
- Then convert AST into ST
- Use top to bottom recursive parsing algorithm to parse the tokens
- To avoid left recursion all the grammar which contains left recursion have converted to right recursive version.
 - Ex: $A \rightarrow A+At$ is converted to $A \rightarrow AtA'$ and $A' \rightarrow +A'$

```
class AstNode {
private:
    Type type;
    AstNode* left{};
    AstNode* right{};
    AstNode* next{};
    const void* value;
    rpal::token::Token* token{};

public:
    explicit AstNode(Type type);
    AstNode(Token* token, Type type);
    AstNode(Type type, const void* value);
    AstNode(Token* token, Type type, const void* value);
    AstNode(Type type, AstNode* left, AstNode* right);
    AstNode(Token* token, Type type, AstNode* left, AstNode* right);
    ~AstNode();
};
```

```
AstNode* Parser::parse_A() {
    Token t = getToken();
    if (t == operators::add()) {...}

    if (t == operators::sub()) {...}

    auto* node_At = parse_At();
    return parse_A_(node_At);
}

AstNode* Parser::parse_A_(AstNode* left) {
    Token t = getToken();
    if (t == operators::add()) {...}

    if (t == operators::sub()) {...}

    return left;
}
```

CSE Machine

Source files: **src/cse.hpp** and **src/cse.cpp**

- Contains the CSE machine
- Interpret the ST

```
class Cse {
private:
    Cse* parent;
    std::vector<AstNode*> original;
    std::stack<AstNode*> control;
    std::stack<AstNode*> stack;
    std::unordered_map<std::string, rpal::parser::AstNode*> env;

public:
    explicit Cse();
    explicit Cse(Cse* parent);
    AstNode* get_env(std::string &key);
    AstNode* read_control();
    Cse* set_parent(Cse* p);
    Cse* get_parent();
    Cse* copy();
    bool empty();
};
```

How to use

> Please Note that string literal syntax should use double quote instead of single quote.

Binary files are in "bin" directory.

Windows:

rpal.exe <source-file-path> => execute code
rpal.exe -t <source-file-path> => linear token list
rpal.exe -a <source-file-path> => abstract syntax tree
rpal.exe -s <source-file-path> => standardized syntax tree

Linux:

rpal <source-file-path> => execute code
rpal -t <source-file-path> => linear token list
rpal -a <source-file-path> => abstract syntax tree
rpal -s <source-file-path> => standardized syntax tree

Example in Linux:

Source Code:

```
let Sum(A) = Psum (A, Order A )  
where rec Psum (T,N) = N eq 0 -> 0 | Psum(T,N-1)+T N  
in Print ( Sum (1,2,3,4,5) )
```

Generate Token list:

```
→ rpal-interpreter (main) × ./bin/rpal -t tests/add  
let  
Sum  
(  
A  
)  
=  
Psum  
(  
A  
,  
Order  
A  
)
```

Generate AST then ST:

```
→ rpal-interpretor (main) × ./bin/rpal -a tests/add
let
.fcn_form
..<ID:Sum>
..<ID:A>
..where
...gamma
....<ID:Psum>
....tau
.....<ID:A>
.....gamma
.....<ID:Order>
.....<ID:A>
...rec
....fcn_form
.....<ID:Psum>
.....,
.....<ID:T>
.....<ID:N>
.....~>
.....eq
.....<ID:N>
.....<INT:0>
.....<INT:0>
.....+

```

```
→ rpal-interpretor (main) × ./bin/rpal -s tests/add
gamma
.lambda
..<ID:Sum>
..gamma
...<ID:Print>
...gamma
....<ID:Sum>
....aug
.....aug
.....aug
.....aug
.....aug
.....aug
.....nil
.....<INT:1>
.....<INT:2>
.....<INT:3>
.....<INT:4>
.....<INT:5>
.lambda
..<ID:A>
..gamma
...lambda
....<ID:Psum>
....gamma
.....<ID:Psum>
.....aug
.....aug
.....nil
.....<ID:A>

```

Execution:

```
→ rpal-interpretor (main) × ./bin/rpal tests/add
15

```

How to build from source:

Requirements

- CMake 3.2+ <https://cmake.org/download/>
- Make <https://gcc.gnu.org/>
- C++ compiler (C++ STD 20) <https://gcc.gnu.org/>

Steps

1. Create directory named "build" `mkdir build` or `md build`
2. Go into that directory `cd build`
3. Type command `cmake ..`

4. Type command ``make``

5. Binary file will be in the build directory ``rpal.exe`` (in windows)